

Программирование на Python для бизнес-аналитики
Магистратура «Бизнес-аналитика и цифровые решения в финансах» · ВШБ НИУ ВШЭ

Соединение, агрегирование и изменение формы табличных данных

Дмитрий Тихомиров · 19 сентября 2026

Проверочная работа № 1

10 минут · окончание в 14:50

Без компьютера, сети, ИИ и конспектов

Темы 01–02 и первый семинар

Сегодняшние лекции не спрашиваются

Вес — 2,5 % итоговой оценки. Совпала буква варианта с соседом — поднимите руку.

Пропустили по уважительной причине — отработка по §8 ПУД: в течение двух недель, по изоморфному варианту, в часы консультаций, не позже 02.10. Без уважительной причины — 0.

Соединение не склеивает таблицы.

Оно их **перемножает**.

Три числа с первого семинара

Как считали	Строк	Выручка, ₽	
Только payments, status = 'success'	3 005 335	880 737 565	×1
+ subscription_members, left join	5 912 729	1 832 324 171	×2,08
+ subscription_members, inner join	5 011 907	1 568 548 593	×1,78

Тип соединения стоит в таблице не для красоты: без него число не воспроизводится.

На семинаре выручка раздулась. Сегодня вопрос: **почему множитель именно 2,08 — а не 3 и не 1,5?**

Кадры из базы через `pd.read_sql`; строки и сумма по payments совпали с SQL. Замер 14.09, окружение курса: pandas 3.0.5, без pyarrow.

Множитель считается до соединения

```
k = members.groupby("subscription_id").size()
k.value_counts().sort_index()
```

```
1    65982
2    66355
3    66111
4    17592
5    17693
Name: count, dtype: int64
```

```
kk = ps["subscription_id"].map(k).fillna(0)
rows = np.maximum(kk, 1).sum()
rub = (ps["amount"] * np.maximum(kk, 1)).sum()
```

- k — участников у подписки:
у 233 733 подписок в среднем **2,378**
- У 100 172 подписок участников нет:
left join оставляет платёж *одной*
строкой — отсюда $\max(k, 1)$
- Платёж размножается на число
участников *своей* подписки

	Строк	₽
Предсказано	5 912 729	1 832 324 171
Факт merge	5 912 729	1 832 324 171

**Совпало до единицы — арифметика,
которую можно сделать заранее.**

Замер 14.09 на всех успешных платежах, окружение курса: pandas 3.0.5, без pyarrow.

Кратность связи: четыре случая

Кратность связи — сколько строк одной таблицы приходится на один ключ другой.

Кратность	Пара в ПРАЙМ	Строк после соединения
1:1	клиенты $\leftrightarrow$ подписки, <i>свёрнутые</i> до клиента	столько же
m:1	payments $\rightarrow$ subscriptions	как слева
1:m	subscriptions $\rightarrow$ subscription_members	больше
m:m	платежи $\leftrightarrow$ tariffs: у тарифа история цен	больше, с обеих сторон

Строк после соединения = $\sum$ по каждому ключу ($n_{\text{слева}} \times n_{\text{справа}}$)
Подписка: 12 платежей $\times$ 3 участника = **36 строк**. Ключ без пары при how="left" даёт множитель 1 — отсюда $\max(k, 1)$.

Виды соединений: что происходит с непарными строками

how=	с парой	непарные слева	непарные справа
"inner"	остаются	выпадают	выпадают
"left"	остаются	остаются, справа NaN	выпадают
"right"	остаются	выпадают	остаются, слева NaN
"outer"	остаются	остаются	остаются

По умолчанию `how="inner"`: непарные строки уходят молча. Поэтому `how=` пишем всегда.

Синтаксис и теория соединений — курс «SQL и базы данных» во втором модуле.
Нам важно не как написать, а сколько строк выйдет.

```
b = pd.merge(
    ps,
    members,
    on="subscription_id",
    how="left",
)
```

- `ps` — успешные платежи, `members` — `subscription_members`
- `on=` — ключ, называем явно
- `how=` — судьба непарных строк

validate=: объявить кратность до соединения

```
pd.merge(ps, members, on="subscription_id", how="left", validate="m:1")
```

```
pandas.errors.MergeError: Merge keys are not unique in right dataset; not a many-to-one merge
```

Duplicates in right:

subscription_id

9103ab23...

9103ab23...

cace3c9b...

cace3c9b...

be02c6e6... ...

- "m:1" — «у платежа не больше одной пары справа», а участников у подписки несколько: **ошибка вместо тихого раздутия**
- Варианты: "1:1", "1:m", "m:1", "m:m". В SQL такой проверки нет
- Проверка идёт до сборки результата, но не мгновенно: 3,7 с, а сам merge — 2,7 с

3 005 335 успешных платежей, замер 14.09, окружение курса: pandas 3.0.5, без pyarrow.

indicator=True: у каждой строки видно, откуда она

```
bi = pd.merge(ps, members, on="subscription_id", how="left", indicator=True)
bi["_merge"].value_counts()
```

```
_merge
both      5011907
left_only  900822
right_only      0
Name: count, dtype: int64
```

both	платёж нашёл участников — по строке на каждого
left_only	подписка без участников. Эти строки inner join выбросил бы молча
right_only	в left join всегда 0: непарные справа туда не попадают

Сверка строк говорит, что число изменилось; indicator= — **какие именно строки** нашли пару.

Замер 14.09, окружение курса: pandas 3.0.5, без pyarrow.

Кратность настоящая — сверните сторону «многие» до зерна

Зерно таблицы — что такое одна её строка: платёж, подписка, клиент.

```
# 1. members: одна строка на участие
# сворачиваем до одной строки на подписку
k = (members
     .groupby("subscription_id", as_index=False)
     .agg(n_members=("client_id", "size")))

# 2. справа теперь один ключ на строку
pay = pd.merge(ps, k, on="subscription_id",
               how="left", validate="m:1")
```

1. Объявите зерно результата
словами: «одна строка — один платёж»
2. Сторону «многие» сверните до него: groupby + agg
3. Соединяйте с validate="m:1":
упадёт, только если ошиблись
вы

Сначала агрегат, потом соединение. Так на семинаре собирается **витрина клиента**.

concat: ВНИЗ, а не вбок — и три тихие ловушки

Две выгрузки за разные месяцы, во второй колонка названа иначе:

```
both = pd.concat([jan, feb])
```

	payment_id	amount	amount_rub
0	a1	199.0	NaN
1	a2	249.0	NaN
2	a3	399.0	NaN
0	b1	NaN	449.0
1	b2	NaN	199.0
2	b3	NaN	249.0

```
pd.concat([jan, feb.rename(columns={"amount_rub": "amount"})], ignore_index=True)
```

Имена приводят до concat, индекс пересобирают: сумма снова 1744.0.

1. **Разные имена — третья колонка и NaN.**
both["amount"].sum() → 847.0, а шесть платежей стоят 1744.0: sum пропускает NaN молча
2. **Индекс повторяется: 0, 1, 2, 0, 1, 2.**
both.loc[0] отдаёт две строки
3. **Целые цены перестают быть целыми.**
int64 после такого concat молча становится float64: NaN в целые не помещается

Второй способ насчитать лишнего

справочник, который живёт во времени

Справочник с историей цен

Цена тарифа меняется, поэтому строк у тарифа несколько — у каждой свой интервал.

row_id	tariff_id	monthly_fee	valid_from	valid_to
1	1	290.0	2025-01-01	2025-03-31
2	1	340.0	2025-03-01	2025-12-31
3	2	590.0	2025-01-01	2025-06-30
4	2	650.0	2025-07-01	2025-12-31

Цены выдуманные. У тарифа 2 цены идут встык, 30.06 и 01.07, — это не пересечение.

Платёж p2 по тарифу 1 прошёл **15 марта 2025, 12:30 UTC**.

Сколько строк вернёт соединение по условию **«цена действовала в момент платежа»**?

validate= проверяет ключ, а не интервал

```
pd.merge(payments_toy, t, on="tariff_id",  
         validate="m:1")
```

→ **MergeError** на обоих справочниках: у тарифа законно несколько цен

```
t.merge(t[["tariff_id", "valid_from"]],  
        on=["tariff_id", "valid_from"],  
        validate="m:1")
```

→ **не упало** и на сломанном: ключ с датой начала уникален, а интервалы всё равно пересекаются

Граница интервала — конвенция курса

```
x = pd.merge(payments_toy, t, on="tariff_id", how="inner", validate="m:m")  
vf = x["valid_from"].dt.tz_localize("UTC")  
vt = (x["valid_to"] + pd.Timedelta(days=1)).dt.tz_localize("UTC")  
x = x[(x["paid_at"] >= vf) & (x["paid_at"] < vt)]
```

valid_to — дата, то есть полночь, а paid_at — момент: отсюда < valid_to + 1 день. Сломанный справочник даёт **8 строк на 6 платежей**, исправный — 6. Без tz_localize: TypeError: Invalid comparison between dtype=datetime64[us, UTC] and DatetimeArray

Игрушечный справочник, окружение курса: pandas 3.0.5, без pyarrow. На живом справочнике ПРАЙМ — на семинаре.

Пересечение ловит самосоединение

```
s = t.merge(t, on="tariff_id",  
            suffixes=("_a", "_b"))  
overlap = (  
    (s["valid_from_a"] <= s["valid_to_b"])  
    & (s["valid_from_b"] <= s["valid_to_a"]))  
s[overlap & (s["row_id_a"] < s["row_id_b"])]
```

→ сломанный: **1 пара** (строки 1 и 2), исправный: 0. Без a < b каждая строка пересекается сама с собой

Протокол сборки: одна строка витрины — это один _____

```
v = step("2. + подписки (агрегат до клиента)", v, owned, on="client_id",  
        validate="1:1", expect_lost=500_000 - 333_905)
```

ПРОТОКОЛ СБОРКИ ВИТРИНЫ

сегмент: вся база

окно фактов: вся история, 2025-01-06...2026-08-30 (UTC)

зерно: одна строка — один client_id

объявлено строк: 500 000

step	rows_before	rows_after	validate	keys_lost	verdict
1. каркас clients	—	500 000	—	—	ok
2. + подписки (агрегат до клиента)	500 000	500 000	1:1	166 095 (33,2 %)	ok, ожидали 166 095
3. + платежи (агрегат до подписки, затем до клиента)	500 000	500 000	1:1	189 825 (38,0 %)	ok, ожидали 189 825
4. + участия (агрегат до клиента)	500 000	500 000	1:1	164 219 (32,8 %)	ok, ожидали 164 219

ЗЕРНО: строк 500 000 = уникальных client_id 500 000 = объявлено 500 000 -> ok

ДЕНЬГИ: витрина 880 737 565,00 Р = контроль по payments 880 737 565,00 Р, разница 0,00 Р -> ok

ЛЮДИ: сумма людей по витрине 445 620 = union владельцев и участников 445 620 -> ok

Одна строка на шаг. Внутри step() — validate=, indicator= и сверка с ожиданием, **объявленным до сборки**. Потеря ключей — не ошибка, если её ждали; вердикт — только «ок», «ок, ожидали ...» или «ВНИМАНИЕ ...»

В ДЗ-2 это 3 балла из 10; без протокола — ноль, даже если числа верные.

Группировка

agg, transform, filter

agg: свернуть до зерна группы

pc — успешные платежи с каналом привлечения владельца: payments → subscriptions → clients, оба шага m:1, строк столько же.

```
pc.groupby("acquisition_channel").agg(  
    revenue=("amount", "sum")
```

acquisition_channel	revenue
crm	109942647.0
organic	244765383.0
partner_telecom	152825375.0
performance	169240793.0
referral	76940661.0
social	109200809.0

Какая форма вернётся

groupby(...)["amount"].sum()	Series (6,)
groupby(...).agg(revenue=...)	(6, 1)
.agg(...).reset_index()	(6, 2)
groupby(..., as_index=False)	(6, 2)

Канал уходит в индекс. Колонкой он станет после `reset_index()` или с `as_index=False`.

Ось: агрегат по умолчанию сворачивает *колонку* — `axis=0`.

Замер 14.09, окружение курса: pandas 3.0.5, без pyarrow.

Сумма по группам не обязана сходиться с общей

```
pc.groupby("acquisition_channel",  
          dropna=False)["amount"].sum()
```

```
acquisition_channel  
crm                109 942 647  
organic            244 765 383  
partner_telecom    152 825 375  
performance        169 240 793  
referral           76 940 661  
social             109 200 809  
NaN                17 821 897  
Name: amount, dtype: float64
```

Сумма по шести каналам	862 915 668 ₽
Все успешные платежи	880 737 565 ₽

Без канала	17 821 897 ₽
60 553 платежа	2,0 %

- groupby не считает пропуск группой — и молчит об этом
- dropna=False возвращает их строкой NaN: снова 880 737 565 ₽
- Это платежи 10 000 клиентов без канала

Сумму по группам сверяют с общей — как строки после соединения.

Тысячи через пробел — настройка `display.float_format`. Замер 14.09, окружение курса: pandas 3.0.5, без pyarrow.

transform: значение группы рядом с каждой строкой

Глагол	Что возвращает	Строк
agg	одно значение на группу	по одной на группу
transform	значение группы — в каждой строке группы	столько же
filter	исходные строки групп, прошедших условие	меньше, но строки те же

b — раздутый left join платежей с участниками. Сколько строк досталось каждому платежу?

```
size = b.groupby("payment_id")["amount"].transform("size")
```

```
shape b (5912729, 9); shape transform (5912729,); dtype int64
```

У платежа подписки с тремя участниками в size стоит 3 — в каждой из трёх его строк. agg("size") дал бы то же число, но одной строкой на платёж.

agg меняет число строк, **transform — нет.**

transform разворачивает двойной счёт обратно

```
share = b["amount"] / b.groupby("payment_id")["amount"].transform("size")  
share.sum()
```

```
880737565.0
```

```
assert abs(share.sum() - control) < 0.01
```

- b — 5 912 729 строк, двойной счёт внутри
- Каждая строка получает свою долю платежа: при трёх участниках — треть суммы
- Сумма долей — снова **880 737 565**, а участники остались в таблице
- agg свернул бы строки и потерял участников: делить надо **рядом со строкой**
- control — сумма успешных платежей без соединения
- Деньги сравнивают с допуском: float64 хранит копейки неточно

transform на 5,9 млн строк — 10,2 с. Замер 14.09, окружение курса: pandas 3.0.5, без pyarrow.

filter: группа проходит или выпадает целиком

```
loyal = ps.groupby("subscription_id").filter(lambda g: len(g) >= 12)
mask = ps.groupby("subscription_id")["payment_id"].transform("size") >= 12
```

117 356 подписок с 12 и более
успешными платежами

1 951 053 их платежа в loyal

- Условие проверяется на *группе*, а возвращаются *исходные строки*: у loyal те же колонки, что у ps
- Строка не проходит сама по себе — только вместе со всей своей подпиской
- ps[mask] через transform — те же самые строки

agg сворачивает, transform подписывает, filter отбирает.

Успешные платежи, замер 14.09, окружение курса: pandas 3.0.5, без pyarrow; оба способа дали одни и те же строки.

Форма выбирается под читателя

широкая — для человека, длинная — для кода

`pivot_table`: месяц × тариф

```
x = pc.assign(month=pc["paid_at"].dt.tz_localize(None).dt.to_period("M"),
               tariff_name=pc["tariff_id"].map(names))
wide = x.pivot_table(index="month", columns="tariff_name", values="amount", aggfunc="sum")
```

`wide.shape` → (20, 2); фрагмент 2026-01...04:

tariff_name month	Базовый	Расширенный
2026-01	21 083 453	25 822 482
2026-02	19 549 959	23 949 576
2026-03	27 710 961	30 518 081
2026-04	25 558 356	28 228 181

`tz_localize(None)` перед `to_period`: без него pandas предупреждает, что зона потеряется.

Разложить рост на «больше платежей» и «дороже платёж» — факторный анализ, тема 06.

Месяц по UTC; тысячи через пробел — `display.float_format`. Замер 14.09, окружение курса: pandas 3.0.5, без `ruarrow`.

- Март к февралю: «Базовый» ×1,42. **Это не только цена**
- Цена 199 → 249 даёт ×1,251; платежей в марте больше в 1,133 раза — в феврале 28 дней
- «Расширенный»: цена 399 → 449 даёт ×1,125, всего ×1,27
- Апрель ниже марта — **дыра в данных** 14–15.04, а не спад. Август неполный

melt: обратно в длинный формат

```
long = wide.reset_index().melt(id_vars="month", value_name="amount")
```

(20, 2) → (40, 3)

	month	tariff_name	amount
0	2025-01	Базовый	10415262.0
1	2025-02	Базовый	12150343.0
2	2025-03	Базовый	14284220.0

	Широкая	Длинная
Зерно	месяц	месяц × тариф
Кому	человеку: отчёт, Excel	коду: groupby, графики
Новый тариф	новая колонка	новые строки

Заказчик присылает широкую таблицу, а groupby и библиотеки графиков хотят длинную. pivot_table и melt — одна операция в две стороны.

Топ-N: «вообще» и «в каждом канале» — разные таблицы

agg — сумма успешных покупок по каналу покупки и сервису, вся история.

```
agg.nlargest(5, "purchases")
```

channel	service_name	purchases
app	Продукты	293 542 575
app	Экспресс-доставка	293 345 829
app	Телемедицина	166 722 399
app	Аптека	166 251 536
web	Продукты	137 349 519

В общем топе-5 нет ни одного pos:
крупные каналы забирают весь рейтинг.

```
(agg.sort_values("purchases", ascending=False)  
 .groupby("channel").head(3))
```

channel	service_name	purchases
app	Продукты	293 542 575
app	Экспресс-доставка	293 345 829
app	Телемедицина	166 722 399
web	Продукты	137 349 519
web	Экспресс-доставка	136 595 057
web	Телемедицина	77 673 257
pos	Экспресс-доставка	76 035 014
pos	Продукты	75 825 406
pos	Аптека	43 367 471

Перепутанные топы — частая ошибка аналитических записок. В Проекте 1 проверьте, какой из двух топов вам нужен.

Рубли без копеек, тысячи через пробел. Замер 14.09, окружение курса: pandas 3.0.5, без pyarrow.

Почему pandas быстрый — и когда перестаёт

```
np.where(ps["payment_method"] == "card", ps["amount"], 0)  
ps.apply(lambda r: r["amount"] if r["payment_method"] == "card" else 0, axis=1)
```

Способ	Время
np.where	0,067 с
.apply(axis=1)	6,67 с
	×99

3 005 335 успешных платежей, медиана из трёх повторов. Сумма одна и та же.

Цена раздутия в памяти: успешные платежи — 832,7 МБ, они же после left join с участниками — 2 075,3 МБ.
По memory_usage(deep=True); замер 14.09 на машине преподавателя, окружение курса: pandas 3.0.5, без pyarrow.

- Колонка — **непрерывный блок памяти одного типа**: np.where уходит одним вызовом в скомпилированный код
- **Распространение форм**: "card" и 0 — одно значение, pandas растягивает его на всю колонку
- .apply(axis=1) собирает объект-строку и вызывает функцию три миллиона раз
- apply законен, когда векторной формы нет

Что унести с этой пары

- **Соединение перемножает:** $\text{строк после} = \sum n_{\text{слева}} \times n_{\text{справа}}$, считается до merge
- **Кратность связи объявляют:** `validate="m:1"` падает, если данные не такие
- **Настоящее «многие» сворачивают до зерна**, потом соединяют
- **`validate` проверяет ключ, а не интервал.** Граница: $\geq \text{valid_from}$ и $< \text{valid_to} + 1 \text{ день}$
- **Протокол сборки:** зерно, деньги, люди — против независимого контроля
- **`groupby` теряет пропуски молча:** `dropna=False` и сверка с общей суммой
- **`agg` меняет число строк, `transform` — нет**

На первом семинаре была примета «сверяйте строки». Сегодня у неё появилась **арифметика**.

К семинару: шесть таблиц витрины клиента

Таблица	Зерно	Что сделать до соединения
clients	клиент	каркас витрины: сворачивать нечего
subscriptions	подписка	свернуть до клиента
payments	платёж	свернуть до подписки, затем до клиента
tariffs	цена в интервале	проверить интервалы, к платежам — по интервалу
subscription_members	участие	свернуть до клиента: флаг «участник»
marketing_spend	канал × день	не присоединяется: другое зерно

Семинар: **23.09** — среда (262), **26.09** — суббота (261).

ДЗ-2 выдаётся **26.09**, дедлайн **04.10**.

Проект 1 выдаётся сегодня

Аналитическая записка · индивидуально · 20 %

Сдача **25.10**, 23:59 · защиты **26.10–06.11**

- Три файла: notebooks/project1.ipynb, записка без кода ЗАПISKA-ПРОЕКТ-1.md или .pdf, ДЕКЛАРАЦИЯ-ИИ-ПРОЕКТ-1.md
- Материал — темы 03–06; факторное разложение — на теме 06
- Защита — по вашему коду: **«не могу объяснить» = «не сделал»**
- Двойной счёт из-за шеринга — среди способов потерять баллы



репозиторий курса →
проекты/проект-1

ДЗ-1 — 27.09, 23:59.
Со снижением — до
04.10.